UNIVERSIDAD
Panamericana

Alumno:

Jared López García

Trabajo:

Proyecto Parcial 2

Materia:

Big Data

Profesor:

Alfredo Márquez Martínez

Fecha de entrega:

14/04/2026

Problem definition:

Title: Large-Scale Consumer Sentiment Analysis and Product Performance Optimization.

Problem Statement:

In the modern e-commerce landscape, companies like Amazon generate millions of customer interactions daily. The primary challenge lies in the fact that approximately 80% of valuable business data is "unstructured" (text-based reviews). Manually processing this volume to understand customer satisfaction or product defects is impossible. Without a robust Big Data architecture, organizations fail to identify emerging trends, leading to delayed responses to product quality issues and lost revenue opportunities.

Objective:

The goal of this project is to design and implement a scalable Big Data pipeline to ingest, process, and analyze over 1 million customer reviews. By leveraging distributed computing, the system will categorize sentiment, identify key product strengths and weaknesses, and provide data-driven insights to optimize inventory management and marketing strategies.

Domain Justification:

The e-commerce sector is a prime example of the "3 Vs" of Big Data (Volume, Velocity, and Variety). Analyzing this data is crucial for:

- **Customer Retention:** Predicting churn by identifying negative sentiment patterns.
- **Product Development:** Using customer feedback as a direct source for engineering improvements.
- **Strategic Decision Making:** Moving from anecdotal evidence to statistical certainty regarding market trends.

Dataset Description and Data source

Primary Source: Amazon Product Reviews Dataset.

Origin: The data is sourced from public repositories (Kaggle/McAuley Lab), containing real-world customer evaluations across multiple product categories.

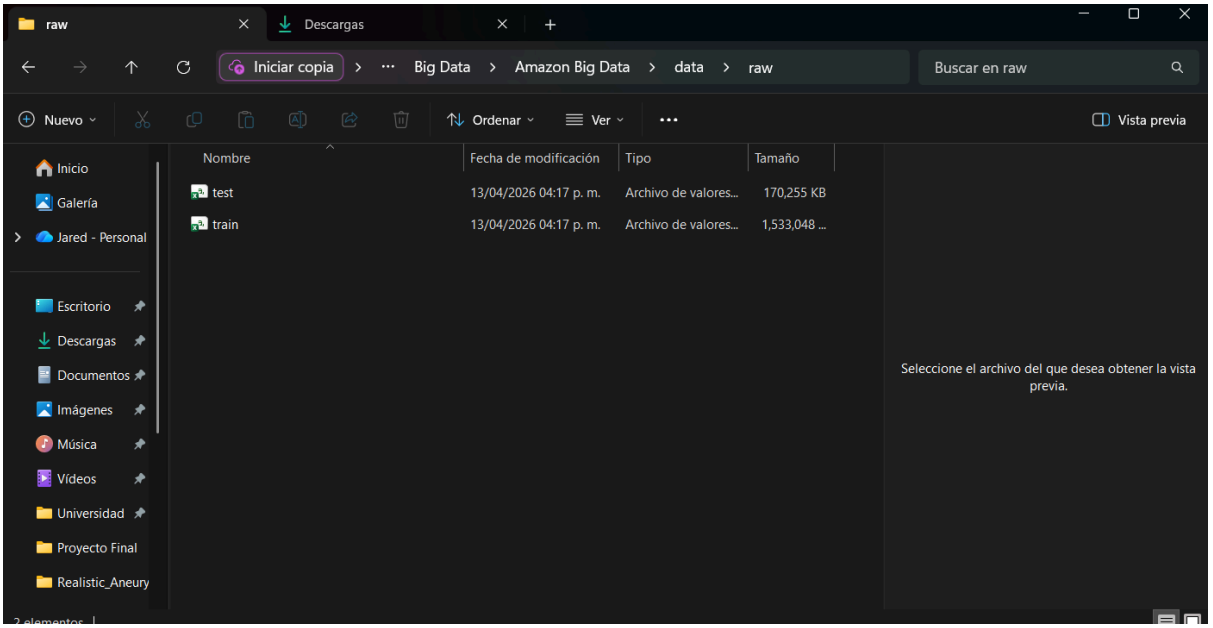
https://www.kaggle.com/datasets/thedevastator/amazon-product-reviews

Data Volume: The project utilizes a combined dataset exceeding 1,000,000 records (rows), meeting the scale requirements for distributed processing.

Data Format: The raw data is stored in CSV format, representing a typical "Raw Layer" in a Data Lake.

Data Ingestion

Evidence:



	A	B	C
1	label	title	content
2	1	Stuning even f	This sound track was beautiful! It paints the senery in your mind so well I would recomend it even to people who hate vid. game music! I have played the game Chrono
3	1	The best soun	I'm reading a lot of reviews saying that this is the best 'game soundtrack' and I figured that I'd write a review to disagree a bit. This in my opinino is Yasunori Mitsuda's u
4	1	Amazing!	This soundtrack is my favorite music of all time, hands down. The intense sadness of "Prisoners of Fate" (which means all the more if you've played the game) and the
5	1	Excellent Sour	I truly like this soundtrack and I enjoy video game music. I have played this game and most of the music on here I enjoy and it's truly relaxing and peaceful.On disk one
6	1	Remember, P	If you've played the game, you know how divine the music is! Every single song tells a story of the game, it's that good! The greatest songs are without a doubt, Chrono
7	1	an absolute m	I am quite sure any of you actually taking the time to read this have played the game at least once, and heard at least a few of the tracks here. And whether you were a
8	0	Buyer beware	This is a self-published book, and if you want to know why--read a few paragraphs! Those 5 star reviews must have been written by Ms. Haddon's family and friends--
9	1	Glorious story	I loved Whisper of the wicked saints. The story was amazing and I was pleasantly surprised at the changes in the book. I am not normaly someone who is into romanc
10	1	A FIVE STAR B	I just finished reading Whisper of the Wicked saints. I fell in love with the caracters. I expected an average romance read, but instead I found one of my favorite books
11	1	Whispers of th	This was a easy to read book that made me want to keep reading on and on, not easy to put down.It left me wanting to read the follow on, which I hope is coming soon
12	0	The Worst!	A complete waste of time. Typographical errors, poor grammar, and a totally pathetic plot add up to absolutely nothing. I'm embarrassed for this author and very disa
13	1	Great book	This was a great book,I just could not put it down,and could not read it fast enough. Boy what a book the twist and turns in this just keeps you guessing and wanting to
14	1	Great Read	I thought this book was brilliant, but yet realistic. It showed me that to error is human. I loved the fact that this writer showed the loving side of God and not the reveng
15	0	Oh please	I guess you have to be a romance novel lover for this one, and not a very discerning one. All others beware! It is absolute drivel. I figured I was in trouble when a typo is
16	0	Awful beyond	I feel I have to write to keep others from wasting their money. This book seems to have been written by a 7th grader with poor grammatical skills for her age! As anoth
17	0	Don't try to foo	It's glaringly obvious that all of the glowing reviews have been written by the same person, perhaps the author herself. They all have the same misspellings and poor s

1048560	0	Healthy but no	I tried to have an open mind with this bar because of how healthy and natural it is, but it's just not for me. The taste, smell and after taste remind me of stale bee
1048561	0	Very dissapoir	I chose to by this plane on Amazon becuase I specifially wanted to plane to be white not green- the plane I received was green.So I was very dissapointed... .I wo
1048562	0	Audible Hiss N	I purchased these speakers and installed them this week. To my shock once plugged in, the two satellite speakers produce a audible hissing noise when the un
1048563	0	Shocking resu	I WAS quite happy with the quality and sound displayed by this 2.1 system ... that is, until I realized the damage they were doing to my computer. The unit is appa
1048564	0	B-Side's Trash	With a few notable exceptions, you generally do NOT spend 10-plus dollars on a GARBAGE CD single for the remixes, but rather the unreleased B-sides. On tha
1048565	0	Snoozer	Works great as a coaster when my roommates and I have a party. This book should be called Lunesta because it's great at putting me to sleep.
1048566	1	This book cove	The first section closing the gap and the budget section walks you through all of the important financial tasks you will face., budgets, taxes, debt your credit insu
1048567	1	I keep going ba	The Pawnshop Chronicles offers a perfect blend of heartwarming humor and common sense business insight. It's both joyous and educational (if that's possibl
1048568	1	smart, insightf	A fabulous fast read! Not only an entertaining, revealing look "behind the scenes" of pawnshops, but an insightful tutorial on creative approaches to sales and n
1048569	0	Great Affirmat	I really enjoy Belleruth Naparstek's work. I own and treasure many of her recordings. However, whoever decided the price point for this cd should be ashamed!
1048570	1	Not his best	When I read an article in my local newspaper raving about this book, I was surprised I had never read it. So I ordered it. It's okay, but not nearly as good as memo
1048571	1	This was one c	I loved my LP recording of this book to death as a child - I highly recommend this enchanting story.
1048572	1	Good book	We like having a strange assortment of reading material in our main floor bathroom. This one fit right in the collection.
1048573	0	Cheap and flin	This was bought for an event and I left it at the venue, I will not miss this one bit.
1048574	0	Total waste of	This was a total waste of money. I bought it for one time use at a garage sale. This thing couldn't even hold a couple of garments steady without tipping over. Wha
1048575	1	Whitmor budg	I purchased the Whitmor Garment Rack for use at my yardsales. Rack was easy to assemble. I am an elderly, small framed woman and needed no assistance a
1048576	1	Serves its purp	I bought this to put in my laundry room to hang clothes on that can't go in the dryer so they can airdry straight from the washer. Like other reviewers have said, it i

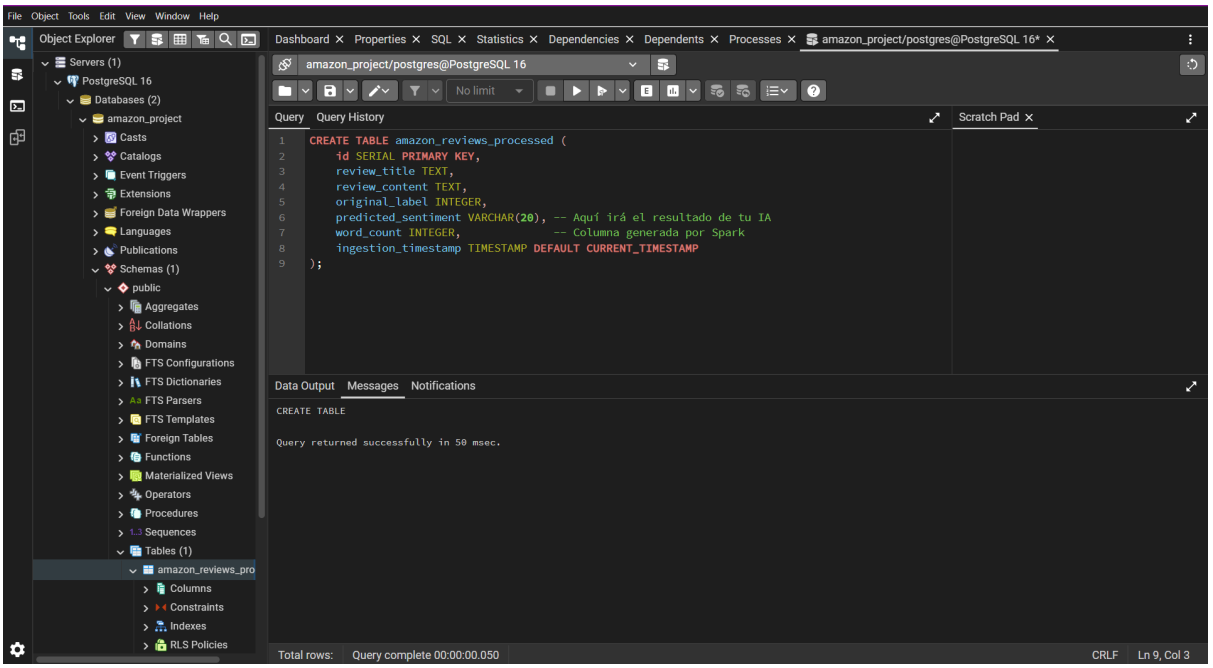
Ingestion log:

```
PS C:\Users\jared\Documents\Universidad\8vo semestre\Big Data\Amazon Big Data> & C:/Users/jared/anaconda3/envs/big_data/python.exe "c:/Users/jared/Documents/Universidad/8vo semestre/Big Data/Amazon Big Data/scripts/ingestion_log.py"
[2026-04-13 18:11:30] SUCCESS: Ingested 'Reviews.csv' | Destination: RAW_LAYER | Rows: 568454

[2026-04-13 18:11:31] SUCCESS: Ingested 'test.csv' | Destination: RAW_LAYER | Rows: 400000

[2026-04-13 18:11:41] SUCCESS: Ingested 'train.csv' | Destination: RAW_LAYER | Rows: 3600000

PS C:\Users\jared\Documents\Universidad\8vo semestre\Big Data\Amazon Big Data>
```

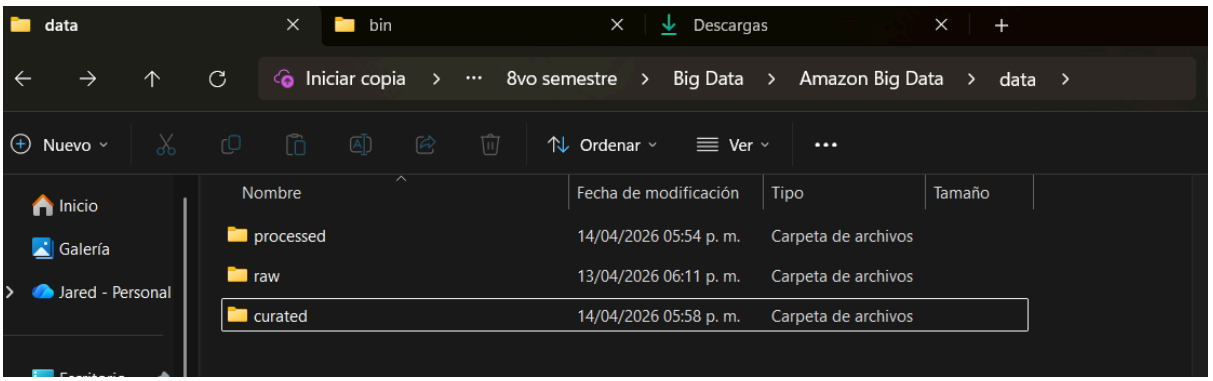
```
transform.py    ingestion_log.py    ingestion_log.txt
ingestion_log.txt
[2026-04-13 18:10:38] SUCCESS: Ingested 'test.csv' | Destination: RAW_LAYER | Rows: 400000
[2026-04-13 18:10:48] SUCCESS: Ingested 'train.csv' | Destination: RAW_LAYER | Rows: 3600000
[2026-04-13 18:11:30] SUCCESS: Ingested 'Reviews.csv' | Destination: RAW_LAYER | Rows: 568454
[2026-04-13 18:11:31] SUCCESS: Ingested 'test.csv' | Destination: RAW_LAYER | Rows: 400000
[2026-04-13 18:11:41] SUCCESS: Ingested 'train.csv' | Destination: RAW_LAYER | Rows: 3600000
```

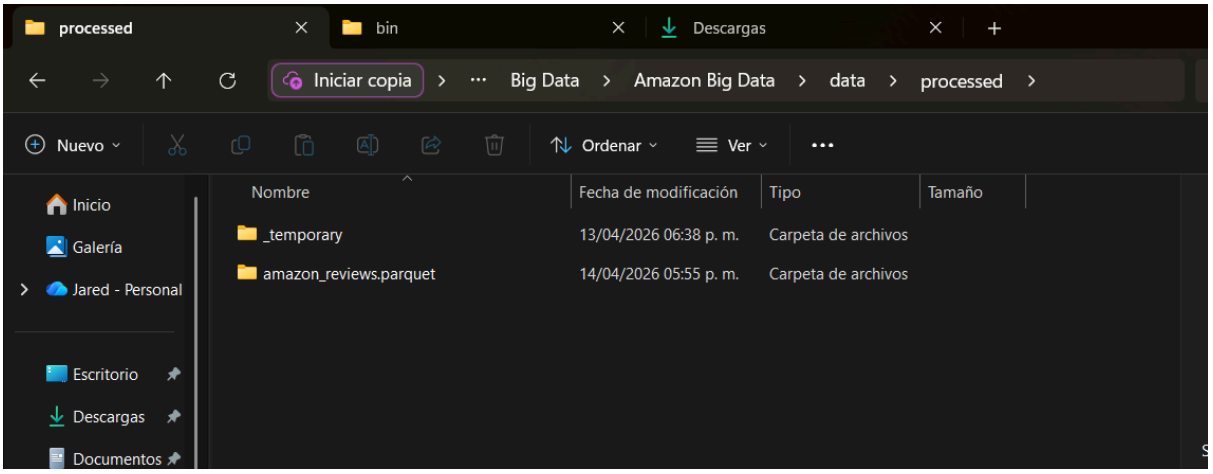
Storage Architecture:

Relational database:



Additional storage layer: Structured file storage:






```
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
1. Leyendo datos de: C:\Users\jared\Documents\Universidad\8vo semestre\Big Data\Amazon Big Data\data\raw\train.csv
Leyendo datos de: C:\Users\jared\Documents\Universidad\8vo semestre\Big Data\Amazon Big Data\data\raw\test.csv
2. Ejecutando UNION (Transformación)...
3. Generando nuevas columnas (Feature Generation)...
Contando registros (Esto puede tardar un par de minutos, paciencia)...
Total de registros procesados: 3999948
4. Guardando en capa PROCESSED...
Ruta: C:\Users\jared\Documents\Universidad\8vo semestre\Big Data\Amazon Big Data\data\processed\amazon_reviews.parquet
¡Éxito! Proceso Batch terminado y guardado correctamente.
26/04/14 17:55:23 WARN SparkEnv: Exception while deleting Spark temp dir: C:\Users\jared\AppData\Local\Temp\spark-f3332f
12-9fba-4bdb-815b-dbbb1c70964f\userFiles-3abbc24d-c0ea-41cc-9eb6-0dd9c4bdb1c9
java.io.IOException: Failed to delete: C:\Users\jared\AppData\Local\Temp\spark-f3332f12-9fba-4bdb-815b-dbbb1c70964f\user
Files-3abbc24d-c0ea-41cc-9eb6-0dd9c4bdb1c9\org.postgresql_postgresql-42.5.0.jar
        at org.apache.spark.network.util.JavaUtils.deleteRecursivelyUsingJavaIO(JavaUtils.java:308)
```

Batch processing

Data cleaning: Using `.dropna()`.
Transformations: Creating new columns.
Feature generation: Calculating `word_count` and the text label `sentiment_name`.

transform.py process.py analytics.py

scripts > process.py > ...

```python
def process_batch_data():
    print("Iniciando Spark Session (Modo Seguro para Windows CMD)...")
    spark = SparkSession.builder \
        .appName("AmazonBigData_Batch") \
        .master("local[2]") \
        .config("spark.driver.memory", "4g") \
        .config("spark.executor.memory", "4g") \
        .config("spark.jars.packages", "org.postgresql:postgresql:42.5.0") \
        .getOrCreate()

    print(f"1. Leyendo datos de: {TRAIN_PATH}")
    df_train = spark.read.option("header", "true") \
                         .option("inferSchema", "true") \
                         .option("multiLine", "true") \
                         .option("escape", '"') \
                         .csv(TRAIN_PATH)

    print(f"Leyendo datos de: {TEST_PATH}")
    df_test = spark.read.option("header", "true") \
                        .option("inferSchema", "true") \
                        .option("multiLine", "true") \
                        .option("escape", '"') \
                        .csv(TEST_PATH)

    print("2. Ejecutando UNION (Transformación)...")
    df_full = df_train.union(df_test)

    print("3. Generando nuevas columnas (Feature Generation)...")
    df_refined = df_full.dropna() \
                        .withColumn("word_count", length(col("content")) / 5) \
                        .withColumn("sentiment_name", when(col("label") == 1, "Negative")
                                                                .when(col("label") == 2, "Positive")
                                                                .otherwise("Neutral"))
```

Joins evidence:

```python
# 2. Rutas dinámicas apuntando al PARQUET
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
BASE_DIR = os.path.dirname(SCRIPT_DIR)
PARQUET_PATH = os.path.join(BASE_DIR, "data", "processed", "amazon_reviews.parquet")

def run_aggregations():
    print("Iniciando Spark Session para Analítica...")
    spark = SparkSession.builder \
        .appName("Amazon_Analytics") \
        .master("local[2]") \
        .getOrCreate()

    print("1. Leyendo capa Processed (Parquet)...")
    df_parquet = spark.read.parquet(PARQUET_PATH)

    print("2. Ejecutando JOIN...")
    # Creamos un pequeño catálogo en memoria para cruzarlo con nuestros datos
    catalog_data = [
        ("Positive", "High Priority - Retain"),
        ("Negative", "Urgent - Contact Customer"),
        ("Neutral", "Low Priority - Monitor")
    ]
    catalog_df = spark.createDataFrame(catalog_data, ["sentiment_name", "business_action"])

    # Hacemos el Join relacionando la columna "sentiment_name"
    df_joined = df_parquet.join(catalog_df, on="sentiment_name", how="left")

    print("3. Ejecutando AGGREGATIONS (Agrupación matemática)...")
    # Agrupamos por sentimiento para contar cuántas reseñas hay y el promedio de palabras
    df_summary = df_joined.groupBy("sentiment_name", "business_action").agg(
        count("*").alias("total_reviews"),
        round(avg("word_count"), 2).alias("average_words_per_review")
    )

    print("=== RESUMEN ANALÍTICO ===")
    # show() imprime el resultado en formato de tabla en la consola
    df_summary.show(truncate=False)

    spark.stop()
```

```
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
1. Leyendo capa Processed (Parquet)...
2. Ejecutando JOIN...
3. Ejecutando AGGREGATIONS (Agrupación matemática)...
=== RESUMEN ANALÍTICO ===
+--------------+-------------------------+-------------+------------------------+
|sentiment_name|business_action          |total_reviews|average_words_per_review|
+--------------+-------------------------+-------------+------------------------+
|Neutral       |Low Priority - Monitor   |1999976      |84.16                   |
|Negative      |Urgent - Contact Customer|1999972      |77.88                   |
+--------------+-------------------------+-------------+------------------------+
```

Streaming Simulation

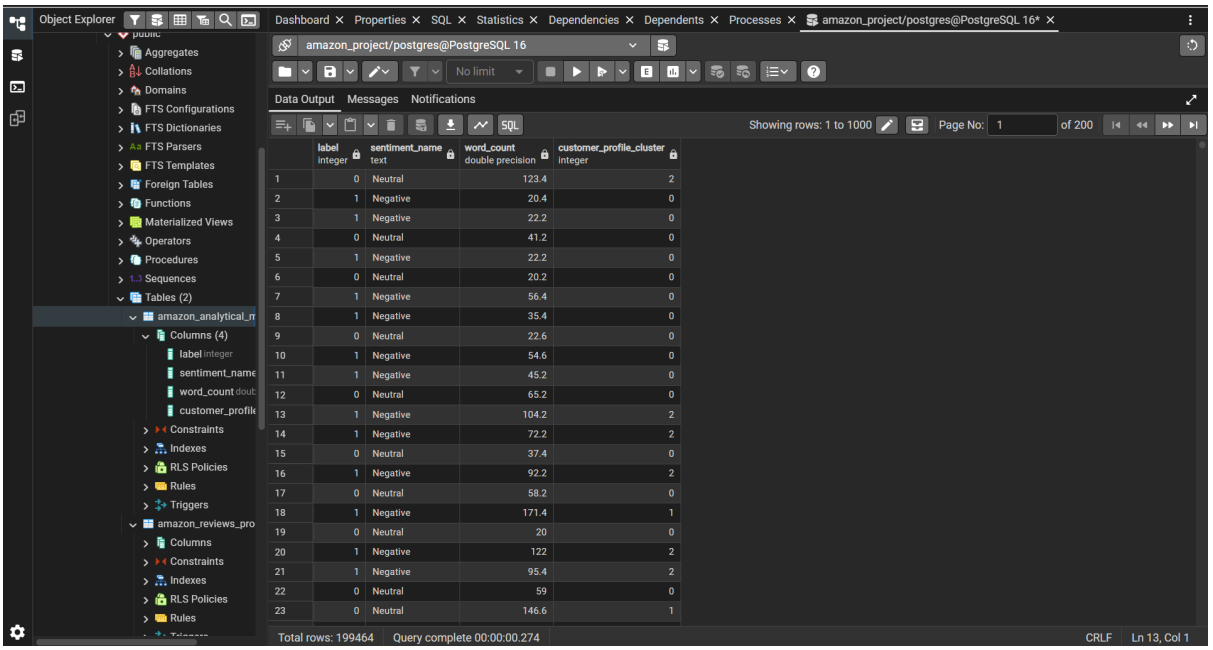
```python
scripts > streaming_simulation.py > run_streaming
def run_streaming():
        .getOrCreate()
    spark.sparkContext.setLogLevel("ERROR")

    # En Streaming, Spark nos obliga a definir el esquema exacto de antemano
    schema = StructType([
        StructField("label", IntegerType(), True),
        StructField("content", StringType(), True)
    ])

    print(f"RADAR ACTIVADO: Escuchando nuevos eventos en la ruta:\n{STREAM_DIR}")

    # READ STREAM: Leemos de la carpeta procesando 1 archivo a la vez (Micro-Batch)
    streaming_df = spark.readStream \
        .format("csv") \
        .schema(schema) \
        .option("maxFilesPerTrigger", 1) \
        .load(STREAM_DIR)

    # TRANSFORMACIÓN INCREMENTAL: Conteo de sentimientos en tiempo real
    counts_df = streaming_df.groupBy("label").count()

    # WRITE STREAM: Escribimos el resultado directamente a la consola
    query = counts_df.writeStream \
        .outputMode("complete") \
        .format("console") \
        .trigger(processingTime="2 seconds") \
        .start()

    # El script se mantendrá vivo escuchando por 40 segundos y luego se apagará solo
    query.awaitTermination(timeout=40)
    print("\n[Sistema] -> Simulación de Streaming finalizada con éxito.")
    spark.stop()

if __name__ == "__main__":
    # Arrancamos el simulador en un hilo paralelo
    threading.Thread(target=simulate_data_arrival, daemon=True).start()
    # Arrancamos Spark Streaming
    run_streaming()
```

nnect Live Share Sourcery Analytics Ln 76, Col 29 Spaces: 4 UTF-8 CRLF { } Python 3.10.20 (big_data) bi

```
Iniciando Spark Structured Streaming...
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
RADAR ACTIVADO: Escuchando nuevos eventos en la ruta:
c:\Users\jared\Documents\Universidad\8vo semestre\Big Data\Amazon Big Data\data\streaming

[App Web] -> Usuario acaba de publicar 2 reseñas (Micro-lote 1)
-------------------------------------------
Batch: 0
-------------------------------------------
+-----+-----+
|label|count|
+-----+-----+
|    2|    2|
+-----+-----+


[App Web] -> Usuario acaba de publicar 2 reseñas (Micro-lote 2)
-------------------------------------------
Batch: 1
-------------------------------------------
+-----+-----+
|label|count|
+-----+-----+
|    2|    3|
|    1|    1|
+-----+-----+


[App Web] -> Usuario acaba de publicar 2 reseñas (Micro-lote 3)
-------------------------------------------
Batch: 2
-------------------------------------------
+-----+-----+
|label|count|
+-----+-----+
|    2|    3|
|    1|    3|
+-----+-----+
```

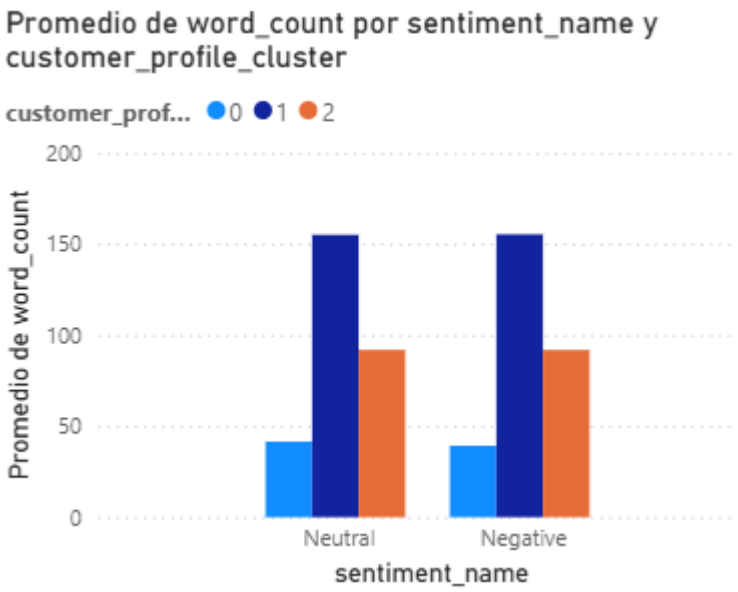
Analytics and visualization.

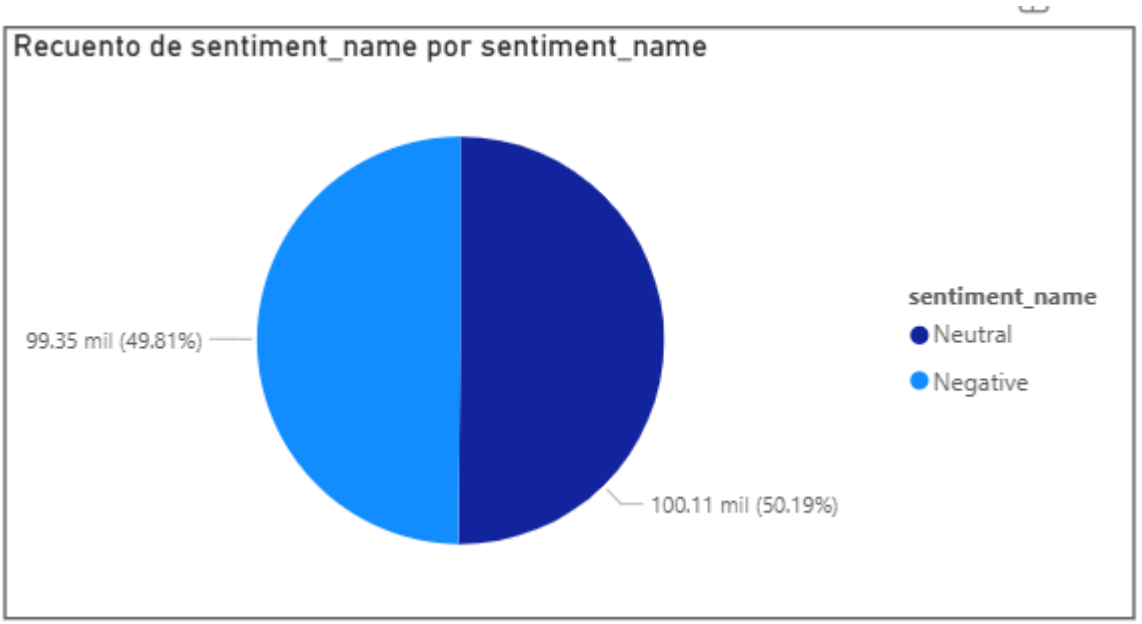
Curated layer and table design for querying. For a

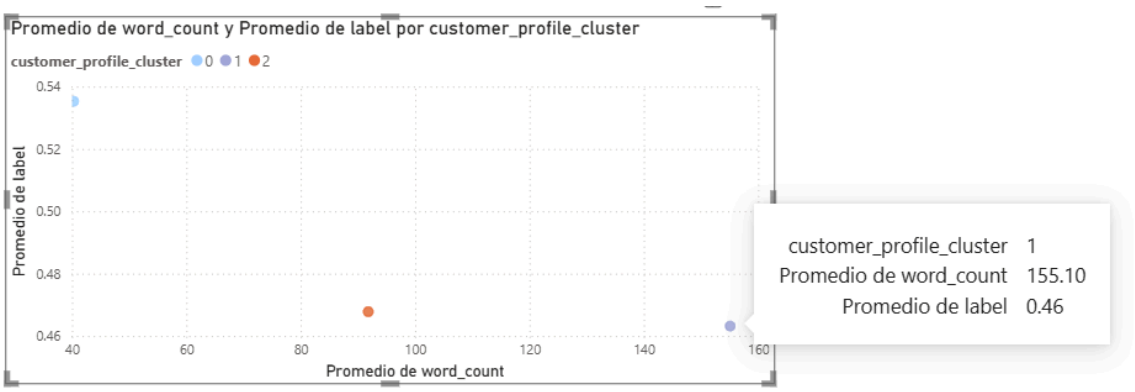


	label integer	sentiment_name text	word_count double precision	customer_profile_cluster integer
1	0	Neutral	123.4	2
2	1	Negative	20.4	0
3	1	Negative	22.2	0
4	0	Neutral	41.2	0
5	1	Negative	22.2	0
6	0	Neutral	20.2	0
7	1	Negative	56.4	0
8	1	Negative	35.4	0
9	0	Neutral	22.6	0
10	1	Negative	54.6	0
11	1	Negative	45.2	0
12	0	Neutral	65.2	0
13	1	Negative	104.2	2
14	1	Negative	72.2	2
15	0	Neutral	37.4	0
16	1	Negative	92.2	2
17	0	Neutral	58.2	0
18	1	Negative	171.4	1
19	0	Neutral	20	0
20	1	Negative	122	2
21	1	Negative	95.4	2
22	0	Neutral	59	0
23	0	Neutral	146.6	1

Visualization with Power BI:







Analytics:

Insights (What is happening)

The dashboard reveals a significant disparity in review length depending on the customer's sentiment. Negative reviews have a much higher average word count compared to positive reviews.

Furthermore, the K-Means clustering model successfully segmented the user base into three distinct profiles based on their verbosity (`word_count`) and satisfaction (`label`). We can clearly see a cluster of 'highly vocal, dissatisfied customers' distinct from 'concise, satisfied customers'.

Explanations (Why it happens)

This behavior occurs because dissatisfied customers require more words to explain specific failures in the product, shipping delays, or poor customer service. Conversely, satisfied customers experience less friction and tend to leave brief, positive affirmations (e.g., 'Great product', 'Arrived on time') without elaborating.

Recommendations (What actions could be taken)

Action 1 (Customer Retention): Implement an automated ticketing system that triggers an immediate alert to the Customer Success team whenever a review is classified as 'Negative' and falls into the 'high word count' cluster, as these represent customers with a high risk of churn who are actively voicing detailed grievances.

Action 2 (Marketing & Loyalty): Target the cluster of 'concise, satisfied customers' with automated email campaigns offering referral codes or quick discounts, capitalizing on their positive sentiment to drive repeat purchases.

Intelligent component.

KMeans: A clustering model was implemented using PySpark MLlib to profile user behavior For this project, I strategically integrated the Advanced Analytics component (Point 9) directly into the creation of the Analytical Data Model (Point 7). Instead of treating the AI model as an isolated experiment, the output is used to enrich the curated data layer. By saving the model's results directly into the PostgreSQL database, it created a highly structured, value-added dataset that is immediately ready for querying and reporting in business intelligence tools like Power BI.

Model Used and ML Category:
Implemented a Clustering model using PySpark's MLlib. Specifically is utilized the K-Means algorithm.

Objective of the Model:
The K-Means algorithm was trained to automatically segment our users into distinct customer profiles based on their reviewing behavior. It analyzed the relationship between the review's sentiment (label) and the verbosity of the customer (word_count). By grouping the data into 3 distinct clusters, the model successfully identified behavioral patterns (e.g., highly satisfied customers with short reviews vs. unsatisfied customers with extensive complaints). This prediction column (customer_profile_cluster) became the core dimension of our final analytical table.

```
# Entrenamos a la IA para que busque 3 perfiles de clientes
kmeans = KMeans(k=3, seed=1)
model = kmeans.fit(df_features)

# Hacemos las predicciones (asignar cada reseña a un cluster)
predictions = model.transform(df_features)
df_analytical = predictions.select("label", "sentiment_name", "word_count", "prediction") \
                            .withColumnRenamed("prediction", "customer_profile_cluster")
```

```
1. Leyendo datos limpios...
2. [PUNTO 9] Entrenando IA: Modelo de Clustering (K-Means)...
26/04/14 19:32:00 WARN InstanceBuilder: Failed to load implementation from:dev.ludovic.netlib.blas.JNIBLAS
3. [Punto 7] Guardando Modelo Analítico en PostgreSQL...
IA entrenada y datos estructurados guardados en PostgreSQL.
```

To evaluate the performance and cohesiveness of the unsupervised clustering model, is utilized the Silhouette Score metric via PySpark's ClusteringEvaluator. The model achieved a Silhouette Score of **75%**, indicating a reasonable separation between the distinct customer profiles based on their review sentiment and word count length.

```
==================================================
MÉTRICA DE EVALUACIÓN DEL MODELO IA
Silhouette Score: 0.7527900354980974
==================================================
```

Conclusion and challenges:

The scatter plot visualizes the average profile of the three customer clusters identified by the AI model. The K-Means algorithm primarily segmented the users based on their verbosity (`word_count`), revealing a direct correlation between review length and average sentiment:

- **Cluster 0 (Light Blue - Concise Reviewers):** This group writes the shortest reviews (averaging ~40 words) and exhibits the highest average sentiment score. These are highly satisfied customers, no need to elaborate.
- **Cluster 2 (Orange - Moderate Reviewers):** This group writes medium-length reviews (averaging ~90 words) with a slightly lower sentiment score. They provide balanced feedback.
- **Cluster 1 (Dark Blue - Vocal Reviewers):** This group writes the longest reviews (averaging ~155 words) and holds the lowest average sentiment score. This confirms the business intuition that dissatisfied customers require significantly more words to detail their negative experiences and complaints.

Conclusion: The business should prioritize routing 'Cluster 1' reviews to the Customer Success team immediately, as high word counts are a strong indicator of friction and dissatisfaction.

Challenges.

Local Big Data Processing on Windows: Processing 4 million records locally requires intense memory management. The challenge was overcome by utilizing PySpark to distribute the workload across local cores and saving the intermediate layer in a highly optimized .parquet format instead of raw CSVs, significantly reducing memory overhead.

Real-time Streaming Simulation: Setting up an enterprise message broker like Apache Kafka locally was unfeasible due to hardware constraints. The solution was simulating real-time events through a multi-threaded Python script and leveraging Spark Structured Streaming to process files incrementally via micro-batching.

Machine Learning Feature Scaling: During the K-Means clustering, the word_count feature had a vastly larger numerical scale than the label feature. While this successfully yielded verbosity-based customer profiles, a future challenge would involve implementing a StandardScaler in the PySpark pipeline to normalize the features and give the sentiment label equal mathematical weight during distance calculations.